SWIGLU AS A LOW-RANK TENSOR MODEL

ANIKET DESHPANDE

In this brief note, I explore a tensor-theoretic interpretation of the transformer FFN, viewing the SwiGLU block as a highly structured low-rank interaction layer over the residual stream. By isolating its first nontrivial multiplicative term, I show that SwiGLU induces a CP-factorized second-order interaction tensor with an implicit diagonal core, which in turn suggests a principled generalization via a learned low-rank Tucker core that allows the model to learn which latent features should interact multiplicatively.

note: this idea is in *very* early stages. I'm actively working on more concrete theory and experiment design and will update the note as I make progress. if you have access to this note and wish to discuss ideas/fixes/implementation, please reach out!

We begin with the residual stream. At a fixed token position, let $x \in \mathbb{R}^d$ denote the residual stream vector entering the feedforward network (FFN) block. In a modern transformer [Vas+17], attention is responsible for moving information across tokens, while the FFN acts pointwise on each token's residual representation. In the architectures of relevance, this pointwise map is usually a gated MLP of the SwiGLU form [Sha20]

$$y = U^T h, \quad h = \left(W^T x\right) \odot \sigma\left(G^T x\right), \tag{1}$$

where $W, G \in \mathbb{R}^{d\times m}$ and $U \in \mathbb{R}^{m\times d}$. Here, W is the main up-projection, G is the gate projection, and U is the down-projection and $h \in \mathbb{R}^m$ is the hidden activation. Writing the j-th columns of W and G as $w_j, g_j \in \mathbb{R}^d$, the j-th hidden coordinate is

$$h_j(x) = \left(w_j^\top x\right) \sigma\left(g_j^\top x\right). \tag{2}$$

Already, we can see a core structural difference between the FFN and the attention map. From the kernel-regression viewpoint, it is Nadaraya-Watson regression with a Gaussian kernel [Nad64, Pai25]. From a statistical mechanical viewpoint, the attention weights form a Boltzmann distribution over key-value pairs,

$$\alpha_i(q) = \frac{\exp\left(\beta q^T k_i\right)}{\sum_j \exp\left(\beta q^T k_j\right)},$$

with inverse temperature $\beta = \tau^{-1}$. The same mechanism therefore interpolates continuously between diffuse averaging at high temeperature and sharp retrieval at low temperature. This dual interpretation is part of attention's tractability. The FFN, specifically the SwiGLU architecture, has no comparable theory. Yet it contains the majority of the model's parameters and is plausibly where many of the remaining gains could lie. I'd like to analyze the FFN from a tensor perspective. Going back to (2), each unit computes a two-feature interaction. It is important to ask, what *kind* of interaction is it, and what structural assumptions does the layer impose on the space of possible feature interactions?

It is useful to expose the interaction structure locally. We can expand the activation σ around a reference point as $\sigma(z) = c_0 + c_1 z + c_2 z^2 + \cdots$. Substituting into (2), we have

$$h_j(x) = (w_j^\top x)\left(c_0 + c_1 g_j^\top x + c_2 (g_j^\top x)^2 + \cdots\right) \tag{3}$$

$$= c_0 w_j^\top x + c_1 (w_j^\top x)(g_j^\top x) + c_2 (w_j^\top x)(g_j^\top x)^2 + \cdots. \tag{4}$$

For sigmoid gating, the first nontrivial interaction appears at bilinear order:

$$h_j(x) = \frac{1}{2}(w_j^\top x) + \frac{1}{4}(w_j^\top x)(g_j^\top x) - \frac{1}{48}(w_j^\top x)(g_j^\top x)^3 + \cdots .$$

For SiLU, which is the defining nonlinearity of the SwiGLU architecture, the leading term is conveniently multiplicative:

$$h_j(x) = \frac{1}{2}(w_j^\top x)(g_j^\top x) + \frac{1}{4}(w_j^\top x)(g_j^\top x)^2 + \cdots . \tag{5}$$

The first term has a clean algebraic interpretation,

$$(w_j^\top x)(g_j^\top x) = x^\top (w_j g_j^\top) x.$$

Because $w_j g_j^\top$ is an outer product, it is a matrix of rank one. So the leading interaction term of each hidden unit is a rank one bilinear form. This is the first key observation. At leading order, SwiGLU builds multiplicative structure out of rank-one separable interactions between pairs of learned scalar projections. Each hidden unit sees two directions in residual space, projects onto them, then multiplies those two scalar responses. This is a very specific inductive bias.

The tensor viewpoint continues to exemplify this. Define the quadratic feature tensor

$$x \otimes x \in \mathbb{R}^{d\times d}, \quad (x \otimes x)_{ab} = x_a x_b.$$

A fully general second-order hidden layer with m interaction channels would require a third-order tensor $\mathcal{T} \in \mathbb{R}^{m\times d\times d}$ with hidden coordinates

$$h_j(x) = \sum_{a=1}^{d}\sum_{b=1}^{d} \mathcal{T}_{jab} x_a x_b = \langle \mathcal{T}_j, x \otimes x\rangle. \tag{6}$$

However, the leading-order SwiGLU interaction does not allow arbitrary slides $\mathcal{T}_{j,:,:}$. Instead, $\mathcal{T}_{j,:,:} = w_j \otimes g_j$, so

$$h_j(x) = \langle w_j \otimes g_j, x \otimes x\rangle = (w_j^\top x)(g_j^\top x). \tag{7}$$

Every hidden slice is forced to factor as an outer product. Now, we include the output projection. At leading order, the output becomes

$$y_o = \sum_{j=1}^{m} U_{jo}(w_j^\top x)(g_j^\top x) = \sum_{a=1}^{d}\sum_{b=1}^{d}\left(\sum_{j=1}^{m} U_{jo} W_{aj} G_{bj}\right) x_a x_b. \tag{8}$$

We define the full second-order interaction tensor as $\mathcal{A} \in \mathbb{R}^{d\times d\times d}$ by $y_o = \sum_{a,b} \mathcal{A}_{oab} x_a x_b$, meaning

$$\mathcal{A}_{oab} = \sum_{j=1}^{m} U_{jo} W_{aj} G_{bj}.$$

This is precisely a canonical polyadic (CP) decomposition [KB09] of the interaction tensor $\mathcal{A}$

$$\mathcal{A} = \sum_{j=1}^{m} u_j \otimes w_j \otimes g_j, \tag{9}$$

where u_j is the output factor associated with the j-th hidden channel. So the leading-order quadratic map induced by SwiGLU is a low-CP-rank tensor model of second-order interactions in the residual stream.

SwiGLU realizes multiplicative feature mixing [JSSB20], but only through sums of rank-one separable factors. Even more strongly, the interaction is coordinate-wise in the hidden space. The i-th feature in one branch only multiplies the i-th feature in the order. In tensor language, the

interaction core is diagonal. This is a useful bias; it gives multiplicative structure at low cost. However, it is also a strong restriction. The model is not learning an arbitrary interaction pattern among latent features; it is learning a diagonal pairing of two projected feature sets.

This naturally suggests a more principled generalization. Instead of forcing the interaction core to be diagonal, we introduce a learned low-rank core tensor. The simplest way to do this is through a Tucker decomposition [KB09]. We choose factor matrices $P, Q \in \mathbb{R}^{d\times r}$ and $R \in \mathbb{R}^{d\times s}$, and a core tensor $\mathcal{G} \in \mathbb{R}^{s\times r\times r}$. We project the residual stream into two latent feature spaces $p = P^\top x \in \mathbb{R}^r$ and $q = Q^\top x \in \mathbb{R}^r$, then define interaction channels as

$$z_\alpha = \sum_{i=1}^{r}\sum_{j=1}^{r} \mathcal{G}_{\alpha ij} p_i \sigma(q_j), \quad \alpha = 1, \ldots, s, \tag{10}$$

and map them back to residual space by $y = Rz$. This is a Tucker-factorized generalization of SwiGLU. In SwiGLU, each projected feature p_i can only interact with the corresponding gated feature q_i. In the Tucker formalism, the learned core $\mathcal{G}$ decides which projected features should interact multiplicatively. Same-index intractions become a special case when $\mathcal{G}$ is superdiagonal. This generalization allows the model to learn richer interaction patterns among latent features, while still maintaining a low-rank structure that keeps the parameter count manageable.

We can directly recover SwiGLU if $r = s = m$ and $\mathcal{G}_{\alpha ij} = \delta_{\alpha i}\delta_{ij}$. Then, $z_i = p_i\sigma(q_i)$, which is exactly the original elementwise gating pattern. So Tucker strictly generalizes SwiGLU rather than replacing it with an unrelated construction.

I believe this direction is promising for three main reasons. First, it is driven by a principled analysis that characterizes the actual interaction structure of the FFN, rather than an ad hoc modification. Second, it introduces a clean tradeoff between expressivity and compute via r and s. The latent rank r and the number of interaction channels s control how much multiplicative structure the block can represent. Third, the contraction should remain reasonably hardware-friendly, since it can be implemented through batched matrix multiplications and small tensor contractions rather than an unstructed dense higher-order tensor.

There is also an interested interpretability motivation. One reason attention has been unusually amenable to mechanistic analysis — and why such analysis has already enabled principled architecture design [Ols+22, Fu+23] — is that its structure is additive and probabilistic: attention heads decompose into weights over values, and those weights often admit clear semantic or algorithmic roles [Ols+22]. The FFN block has not yielded to the same kind of decomposition. One plausible reason is that its interaction geometry is much harder to factor cleanly downstream. If the multiplicative structure inside the FFN were made more explicit and low-rank, then sparse decomposition methods (dictionary learning, SAEs, transcoders) [Bri+23] might have a cleaner object to work with. From a rate-distortion perspective [PKW24], current FFNs may be hiding structure in a way that makes it difficult to disentangle without a more explicit inductive bias. By contrast, the proposed Tucker generalization would encourage the model to learn a more explicit interaction structure, which could in turn facilitate mechanistic interpretability without sacrificing the empirical advantage of multiplicative gating.

Email address: aniket4@illinois.edu

References

[Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017.

[Sha20] N. Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202, 2020.

[KB09] T. G. Kolda and B. W. Bader. *Tensor Decompositions and Applications*. SIAM Review, **51**(3):455–500, 2009.

[Nad64] E. A. Nadaraya. *On Estimating Regression*. Theory of Probability and its Applications, **9**(1):141–142, 1964.

[Pai25] D. Pai. *Regression is All You Need*. Tilde Research, 2025. https://blog.tilderesearch.com/vignettes/regression

[JSSB20] S. M. Jayakumar, J. Menick, W. M. Czarnecki, J. Schwarz, J. Rae, S. Osindero, Y. W. Teh, T. Harley, and R. Pascanu. *Multiplicative Interactions and Where to Find Them*. ICLR, 2020.

[Ols+22] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, et al. *In-context Learning and Induction Heads*. Transformer Circuits Thread, 2022.

[Fu+23] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré. *Hungry Hungry Hippos: Towards Language Modeling with State Space Models*. ICLR, 2023.

[PKW24] D. Pai, A. Keigwin, and A. Wang. *The Rate Distortion Dance of Sparse Autoencoders*. Tilde Research, 2024. https://blog.tilderesearch.com/blog/rate-distortion-saes

[Bri+23] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, et al. *Towards Monosemanticity: Decomposing Language Models with Dictionary Learning*. Transformer Circuits Thread, 2023.